



Adv ML for Gen-AI

Diffusion

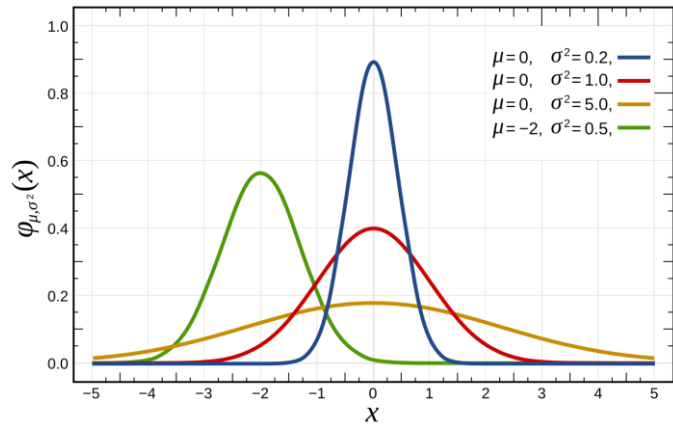
<https://ml-graph.github.io/spring-2026/>

Yu Wang, Ph.D.
Assistant Professor
Department of Computer Science
University of Oregon
Personal: <https://yuwang0103.github.io/>
Lab: <https://kindlab-fly.github.io/>



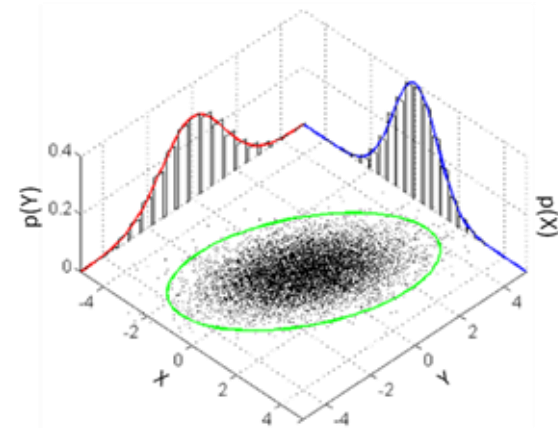
Summary

1D Gaussian Distribution

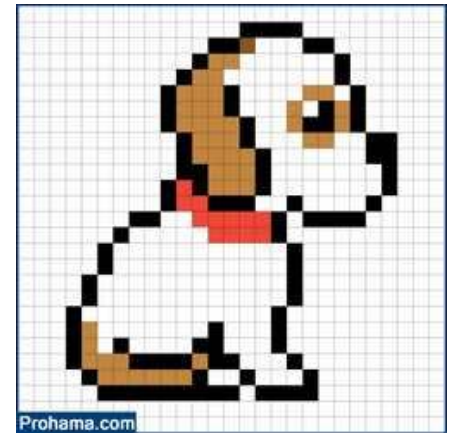
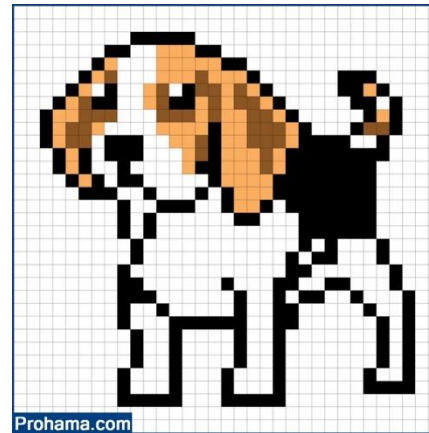
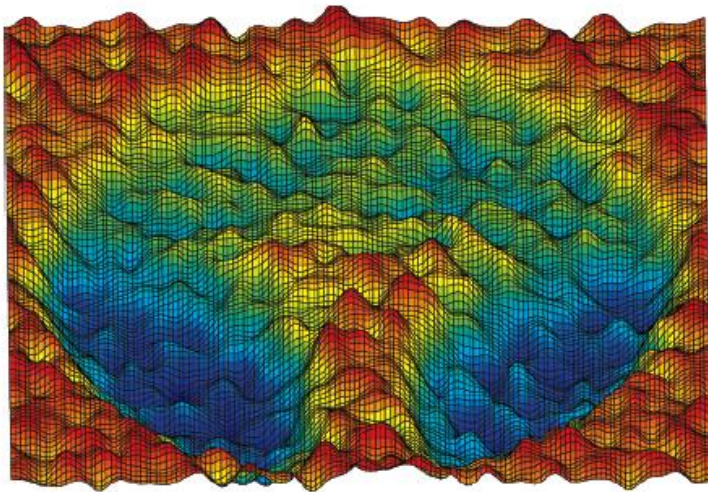


$\mathbb{R}$

2D Gaussian Distribution



$\mathbb{R}^2$



$\mathbb{R}^{256 \times 256}$



Probability distribution of the **objective** based on the **observed data**

- **Machine Learning Methods**

- Gaussian Kernel Density Estimation
- Gaussian Mixture Models



Using **existing function** to estimate what you do not know that can best fit your observation

- **Deep Learning Methods**

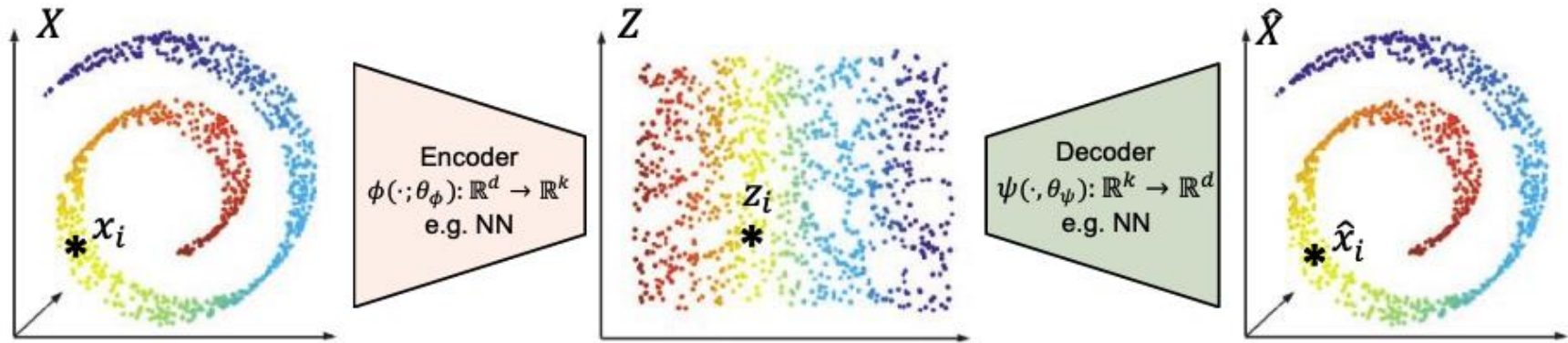
- Auto-Encoder (AE)
- Variational AE (LLM is actually a VAE)
- Generative Adversarial Network
- Diffusion Model

Using **learnable function** to estimate what you do not know that can best fit your observation

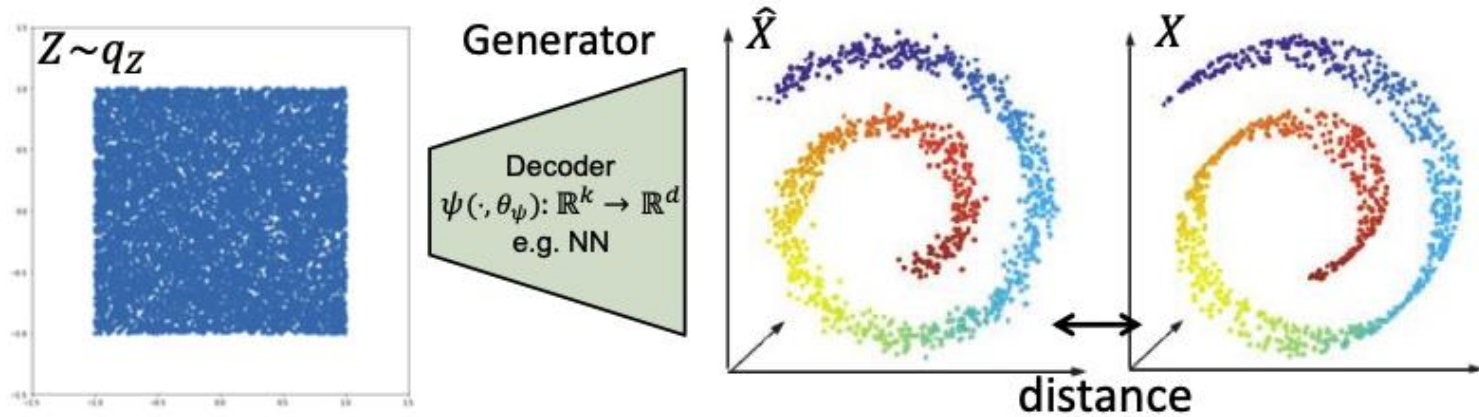


Summary

Auto-encoder based generative modeling



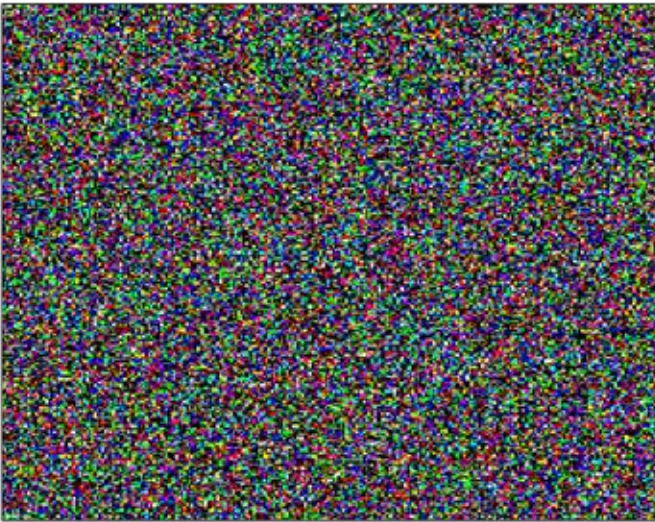
Encoder-less Generative Modeling



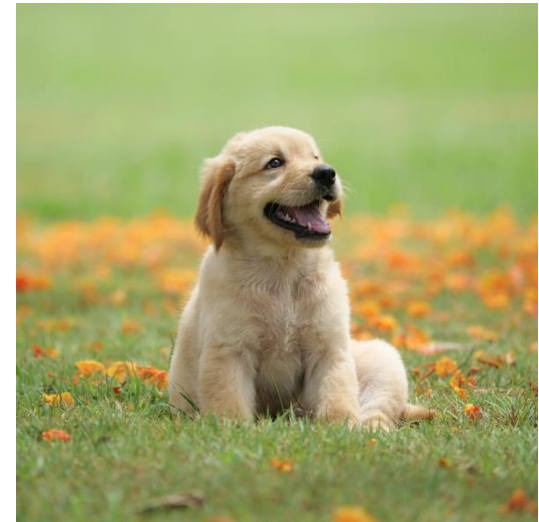


Problem of Previous Generation

Gaussian Noise 256x256 (3-channel)



**One-shot
Generation**



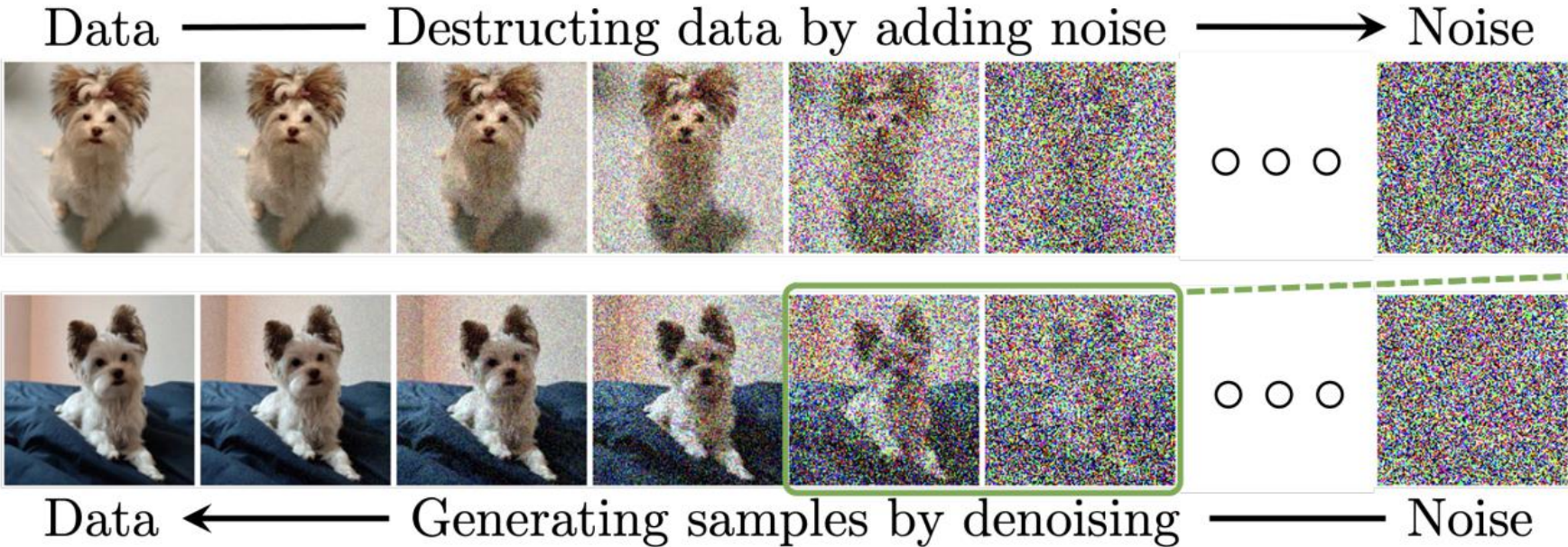
Standard Gaussian



Target Distribution



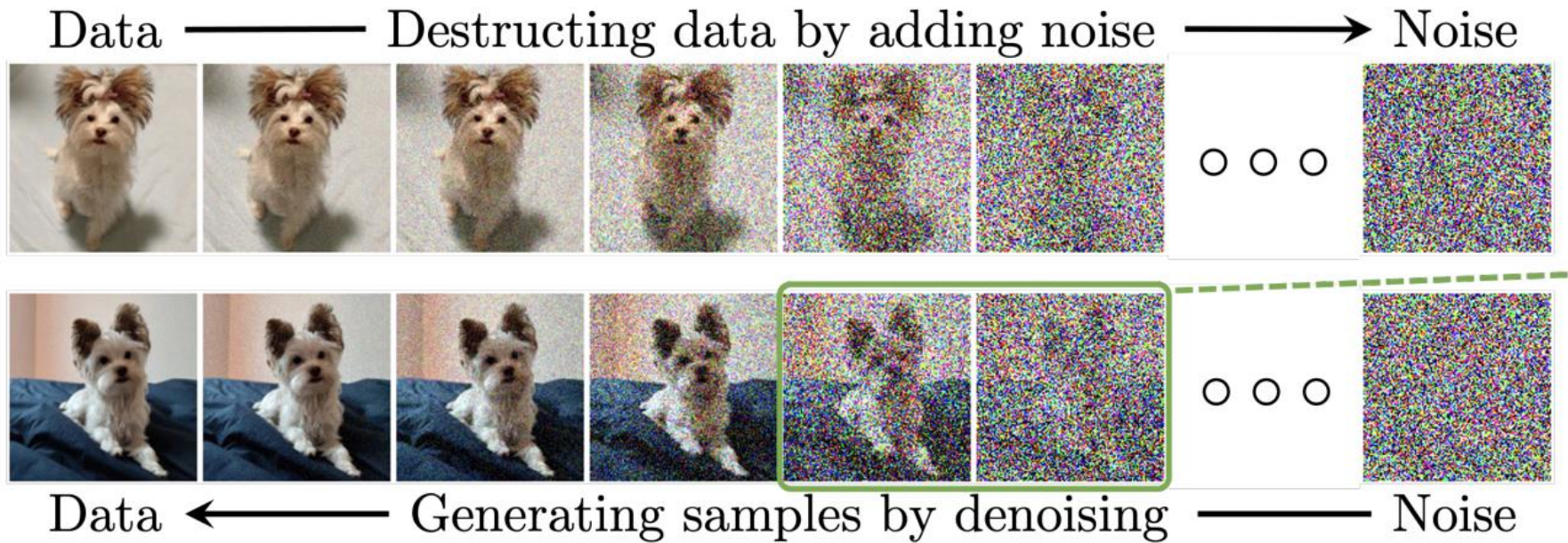
Motivation of Diffusion



Can we construct the image step by step?



Forward/Backward Process of Image Diffusion



- **Forward Process (Data - Noise):** Gradually add noise to generate intermediate data
- **Backward Process (Noise - Data):** **Learn** to denoise the intermediate data and recover back

So we need both noisy image and ground-truth image



Forward Process to obtain noisy image



data distribution $\mathbf{x}_0 \sim q(\mathbf{x}_0)$. $\mathbf{x}_1, \mathbf{x}_2 \dots \mathbf{x}_T$ with transition kernel $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}),$$

$\beta_t \in (0, 1)$ is a hyperparameter

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

Recursively Sampling

$x = \mu + \sigma \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$

Re-parametric trick

$$q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I), \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s).$$

One step Sampling

Recursively expansion



Forward Process to obtain noisy image

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}),$$

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I}).$$

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s).$$

$\beta_t \in (0, 1)$ is a hyperparameter

In Reality

One step Sampling

Forward process parameters

```
def linear_beta_schedule(T, beta_start=1e-4, beta_end=2e-2):
    return torch.linspace(beta_start, beta_end, T)
```

T = 200

```
betas = linear_beta_schedule(T).to(device)
```

```
alphas = 1.0 - betas
```

```
alpha_bar = torch.cumprod(alphas, dim=0)
```

```
print('T:', T)
```

```
print('beta range:', float(betas.min()), '-->', float(betas.max()))
```

```
print('alpha_bar[0]:', float(alpha_bar[0]), 'alpha_bar[-1]:', float(alpha_bar[-1]))
```

Closed-form sampling from $q(\mathbf{x}_t \mid \mathbf{x}_0)$

```
def q_sample(x0, t, noise=None):
```

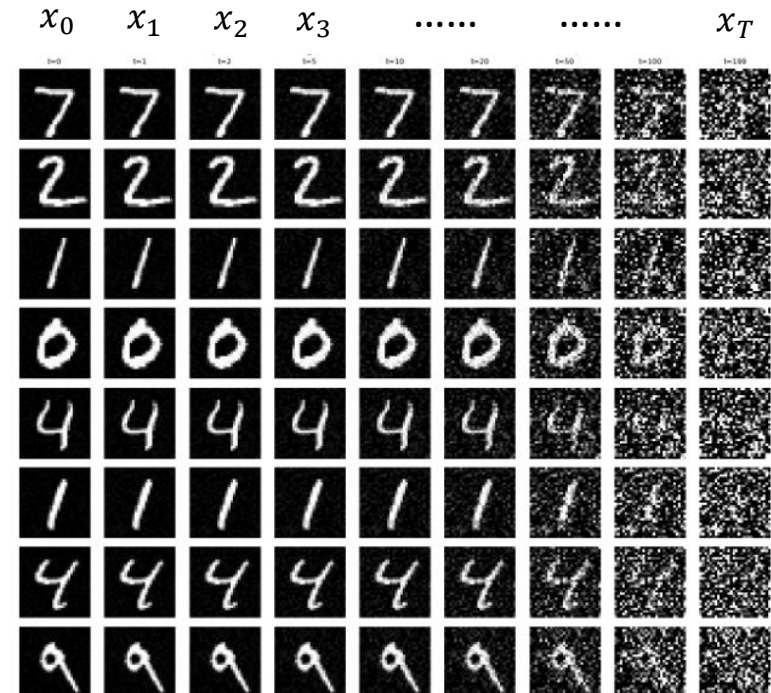
```
    # Sample x_t given x_0 at discrete time index t (0..T-1)
```

```
    if noise is None:
```

```
        noise = torch.randn_like(x0)
```

```
    a_bar = alpha_bar[t].view(-1, 1, 1, 1) # (B,1,1,1)
```

```
    return torch.sqrt(a_bar) * x0 + torch.sqrt(1.0 - a_bar) * noise
```





Forward Process to obtain noisy image

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}),$$

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I}).$$

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s).$$

$\beta_t \in (0, 1)$ is a hyperparameter

In Reality

One step Sampling

Forward process parameters

```
def linear_beta_schedule(T, beta_start=1e-4, beta_end=2e-2):  
    return torch.linspace(beta_start, beta_end, T)
```

```
T = 200
```

```
betas = linear_beta_schedule(T).to(device)
```

```
alphas = 1.0 - betas
```

```
alpha_bar = torch.cumprod(alphas, dim=0)
```

```
print('T:', T)
```

```
print('beta range:', float(betas.min()), '->', float(betas.max()))
```

```
print('alpha_bar[0]:', float(alpha_bar[0]), 'alpha_bar[-1]:', float(alpha_bar[-1]))
```

Closed-form sampling from $q(\mathbf{x}_t \mid \mathbf{x}_0)$

```
def q_sample(x0, t, noise=None):
```

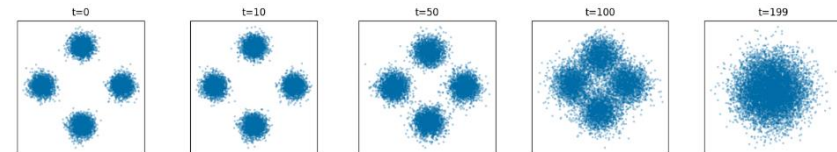
```
    # Sample  $\mathbf{x}_t$  given  $\mathbf{x}_0$  at discrete time index  $t$  ( $0..T-1$ )
```

```
    if noise is None:
```

```
        noise = torch.randn_like(x0)
```

```
    a_bar = alpha_bar[t].view(-1, 1, 1, 1) # (B,1,1,1)
```

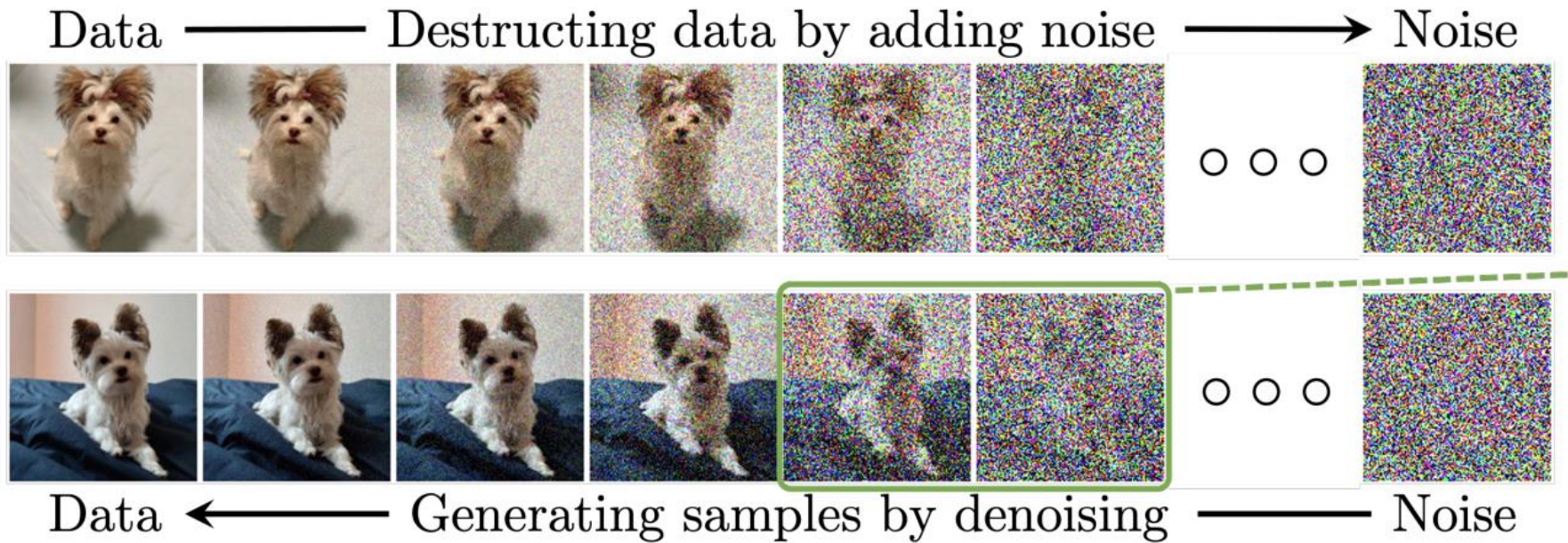
```
    return torch.sqrt(a_bar) * x0 + torch.sqrt(1.0 - a_bar) * noise
```



Once you have a forward process, you naturally have a way to generate a lot of noisy version of your data



Backward Process to learn to reconstruct

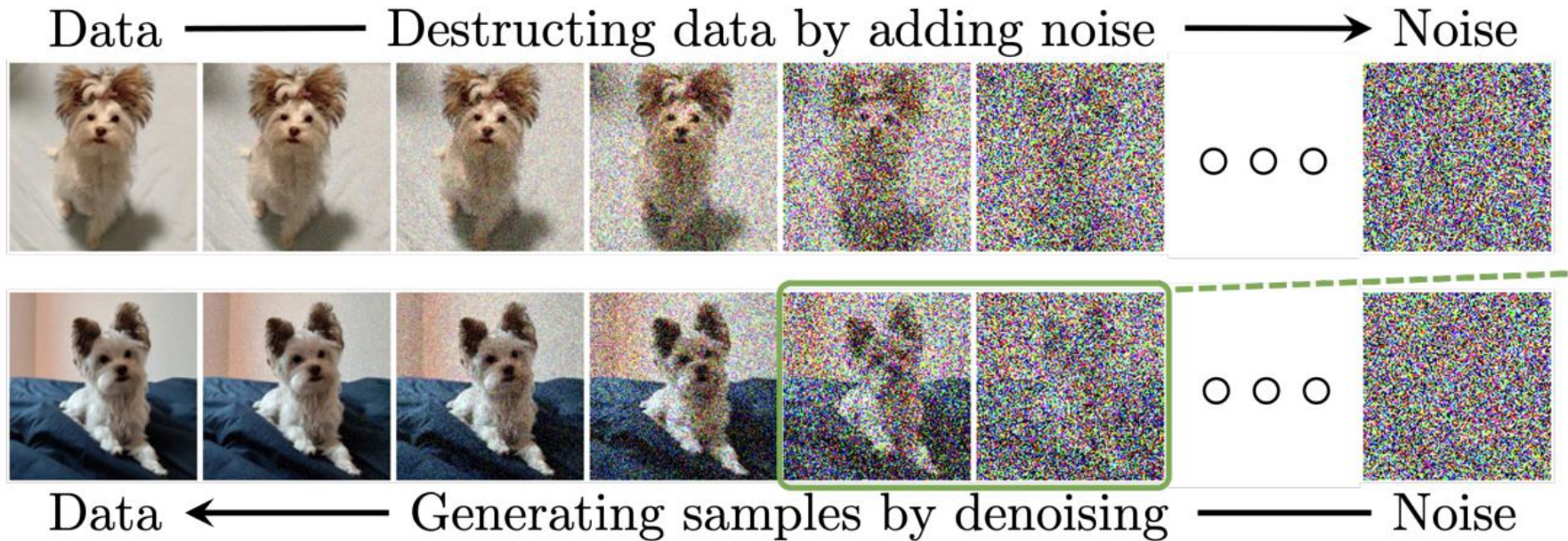


- **Backward Process (Noise - Data):** Learn to denoise the intermediate data and recover back

$$x_t \rightarrow x_{t-1}$$



Backward Process to learn to reconstruct



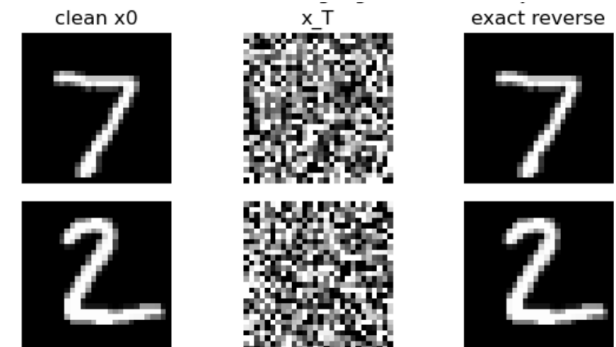
$$x_t \rightarrow x_{t-1}$$

Ideally it works!

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{\beta_t} \epsilon_t,$$

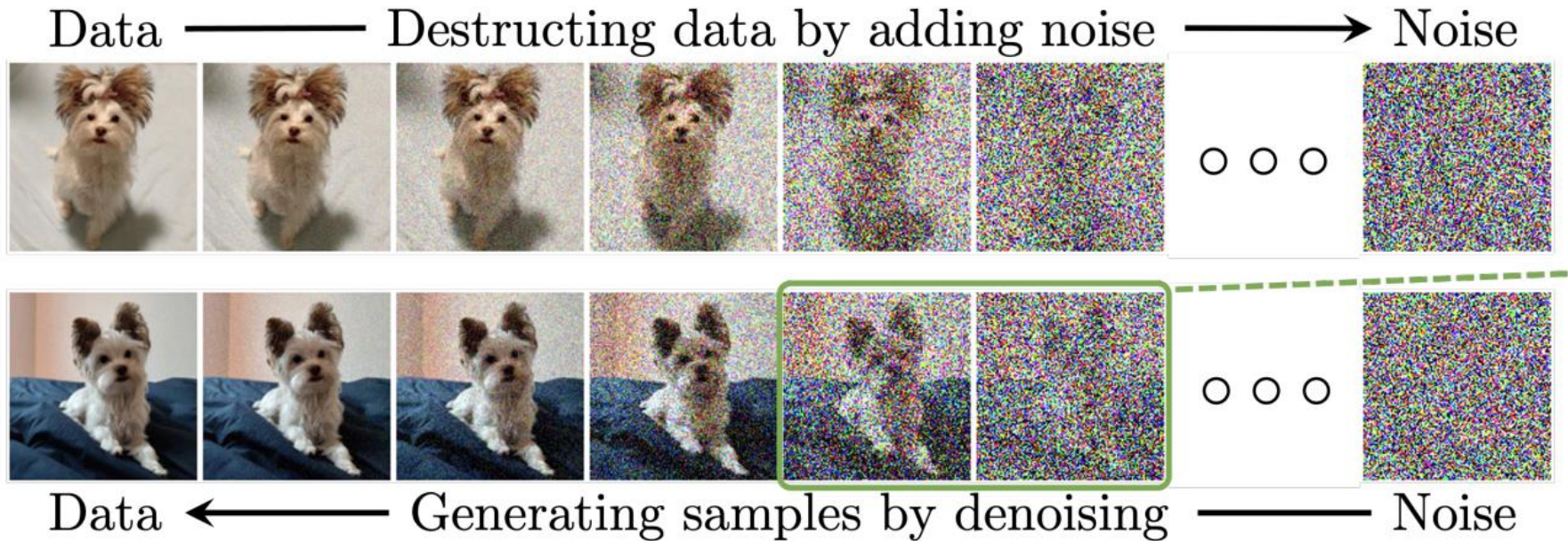
$$x_{t-1} = \frac{x_t - \sqrt{\beta_t} \epsilon_t}{\sqrt{\alpha_t}}.$$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon,$$





Backward Process



$$x_t \rightarrow x_{t-1}$$

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{\beta_t} \epsilon_t,$$

$$x_{t-1} = \frac{x_t - \sqrt{\beta_t} \epsilon_t}{\sqrt{\alpha_t}}. \quad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon,$$

Given x_t , predict something that helps us move back toward data.

There are three possible targets:

$x_t \rightarrow x_{t-1}$	previous noisy state
$x_t \rightarrow x_0$	clean image
$x_t \rightarrow \epsilon$	noise that corrupted the image



Backward Process

Given x_t , predict something that helps us move back toward data.

There are three possible targets:

$x_t \rightarrow x_{t-1}$	previous noisy state
$x_t \rightarrow x_0$	clean image
$x_t \rightarrow \epsilon$	noise that corrupted the image

Forward Process

cat $\longrightarrow$ c#a\$t%

Backward Process

cat $\longleftarrow$ c#a\$t%

What was the exact previous corrupted string?
Was it "c#a\$t%", "ca\$t%", "c#at%", ... ?

$$x_t \rightarrow x_{t-1}$$

What was the clean string?

$$x_t \rightarrow x_0$$

Which symbols look like noise?
Answer: #, \$, %

$$x_t \rightarrow \sigma$$

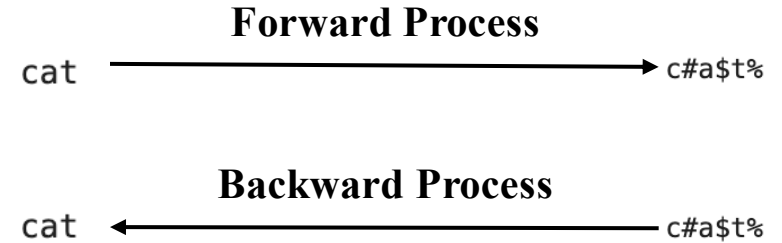


Backward Process

Given x_t , predict something that helps us move back toward data.

There are three possible targets:

$x_t \rightarrow x_{t-1}$	previous noisy state
$x_t \rightarrow x_0$	clean image
$x_t \rightarrow \epsilon$	noise that corrupted the image



What was the exact previous corrupted string?
Was it "c#a\$t", "ca\$t%", "c#at%", ... ?

$$x_t \rightarrow x_{t-1}$$

What was the clean string?

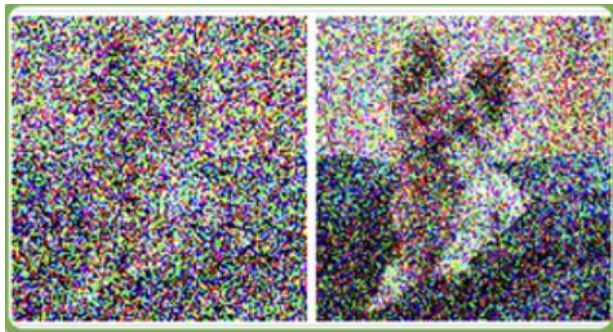
$$x_t \rightarrow x_0$$

Which symbols look like noise?
Answer: #, \$, %

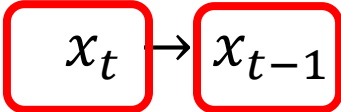
$$x_t \rightarrow \sigma$$



Backward Process – $x_t \rightarrow x_{t-1}$



Current state

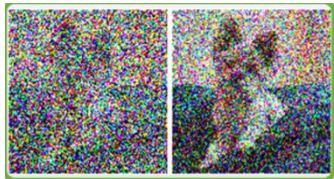
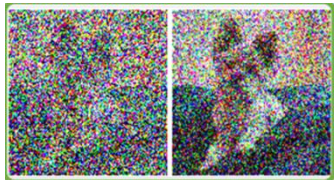


Previous state

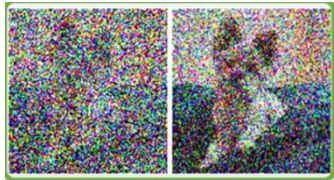
$$\{(x_{t-1}, x_t)^k\}_{k=1}^K$$

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I), \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s).$$



...



$$\sum_{t=1}^T \sum_{k=1}^K \|f_{\theta}(x_t^k) - x_{t-1}^k\|_2^2$$

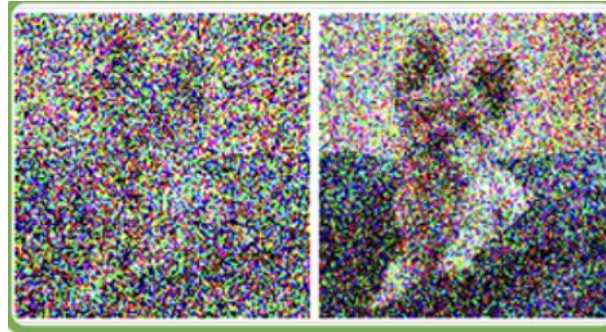
For each minibatch:

1. Sample $t \sim \text{Uniform}\{1, \dots, T\}$. (We never need to predict x_{-1} .)
2. Sample x_{t-1} from the closed form marginal:
3. Take one forward Markov step to get x_t :

Then train $f_{\theta}(x_t, t)$ to match x_{t-1} .



Backward Process – $x_t \rightarrow x_{t-1}$



Current state

x_t

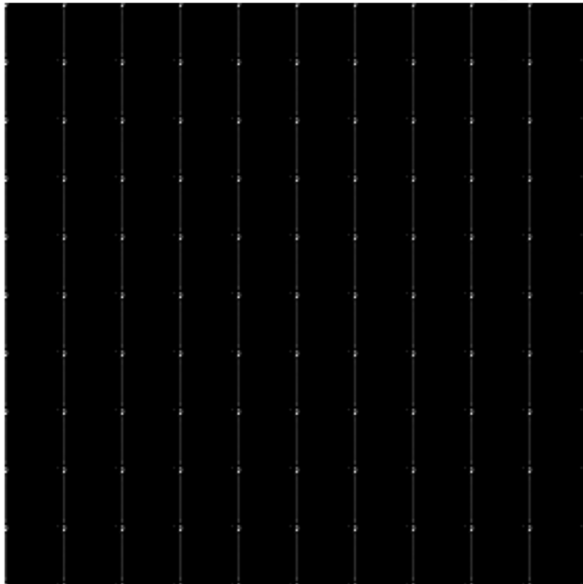
$\rightarrow$

x_{t-1}

Previous state

$\{(x_{t-1}, x_t)^k\}_{k=1}^K$

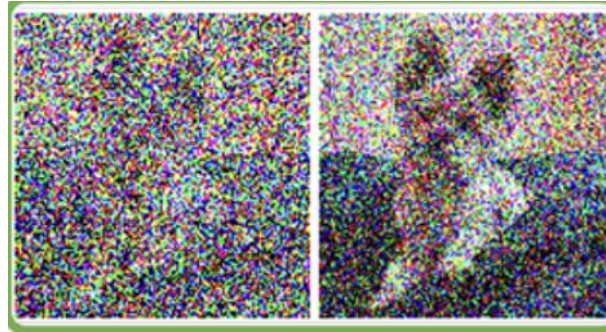
Naive generation (deterministic): repeatedly predict x_{t-1}



Unfortunately it does not work



Backward Process – $x_t \rightarrow x_{t-1}$



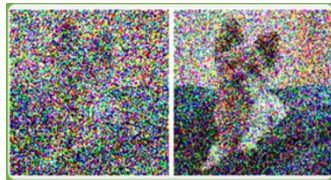
Current state

x_t

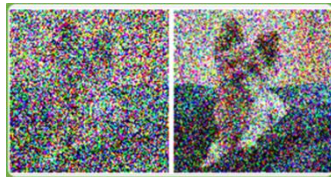
$\rightarrow x_{t-1}$

Previous state

$$\{(x_{t-1}, x_t)^k\}_{k=1}^K$$



...



What was the exact previous corrupted string?
Was it "c#a\$t", "ca\$t%", "c#at%", ... ?

c#a\$t%

$$(x_{t-1}^1, \sigma_{t-1}^1) \rightarrow x_t$$

$$(x_{t-1}^2, \sigma_{t-1}^2) \rightarrow x_t$$

$\vdots \quad \vdots \quad \vdots$

$$(x_{t-1}^k, \sigma_{t-1}^k) \rightarrow x_t$$

**Many previous
timestep,
One current
Timestep Image**

$$\sum_{t=1}^T \sum_{k=1}^K \|f_{\theta}(x_t) - x_{t-1}\|_2^2$$



Backward Process – $x_t \rightarrow x_0$

Given x_t , predict something that helps us move back toward data.

There are three possible targets:

$x_t \rightarrow x_{t-1}$	previous noisy state
$x_t \rightarrow x_0$	clean image
$x_t \rightarrow \text{epsilon}$	noise that corrupted the image

Forward Process

cat $\longrightarrow$ c#a\$t%

Backward Process

cat $\longleftarrow$ c#a\$t%

What was the exact previous corrupted string?
Was it "c#a\$t%", "ca\$t%", "c#at%", ... ?

$$x_t \rightarrow x_{t-1}$$

What was the clean string?

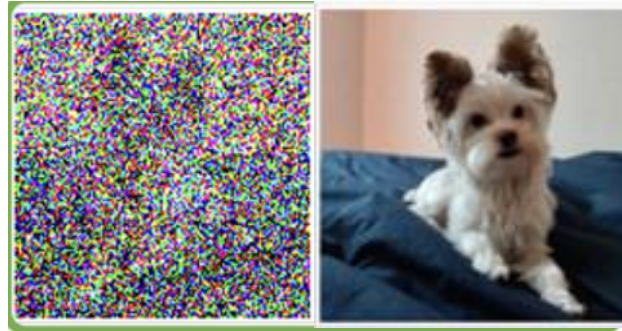
$$x_t \rightarrow x_0$$

Which symbols look like noise?
Answer: #, \$, %

$$x_t \rightarrow \sigma$$



Backward Process – $x_t \rightarrow x_0$



Current state

$x_t \rightarrow x_0$

Initial state

$\{(x_t, x_0)^t\}_{t=1}^T$

This version trains a model

$$g_\theta(x_t, t) \approx x_0.$$

For each minibatch:

1. Sample clean images $x_0 \sim p_{\text{data}}(x)$.
2. Sample timesteps $t \sim \text{Uniform}\{1, \dots, T\}$.
3. Sample Gaussian noise $\varepsilon \sim \mathcal{N}(0, I)$.
4. Create the noisy image directly from the closed form:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon.$$

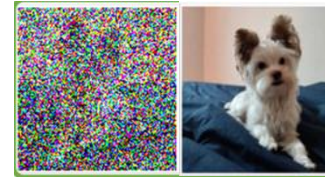
5. Predict the clean image:

$$\hat{x}_0 = g_\theta(x_t, t).$$

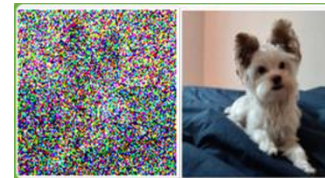
6. Train with reconstruction loss:

$$\mathcal{L}_{x_0}(\theta) = \|g_\theta(x_t, t) - x_0\|^2.$$

After training, the one-step diagnostic is: given a noisy x_t , does the model recover the original clean image x_0 ? not yet a full stochastic DDPM sampler.



...

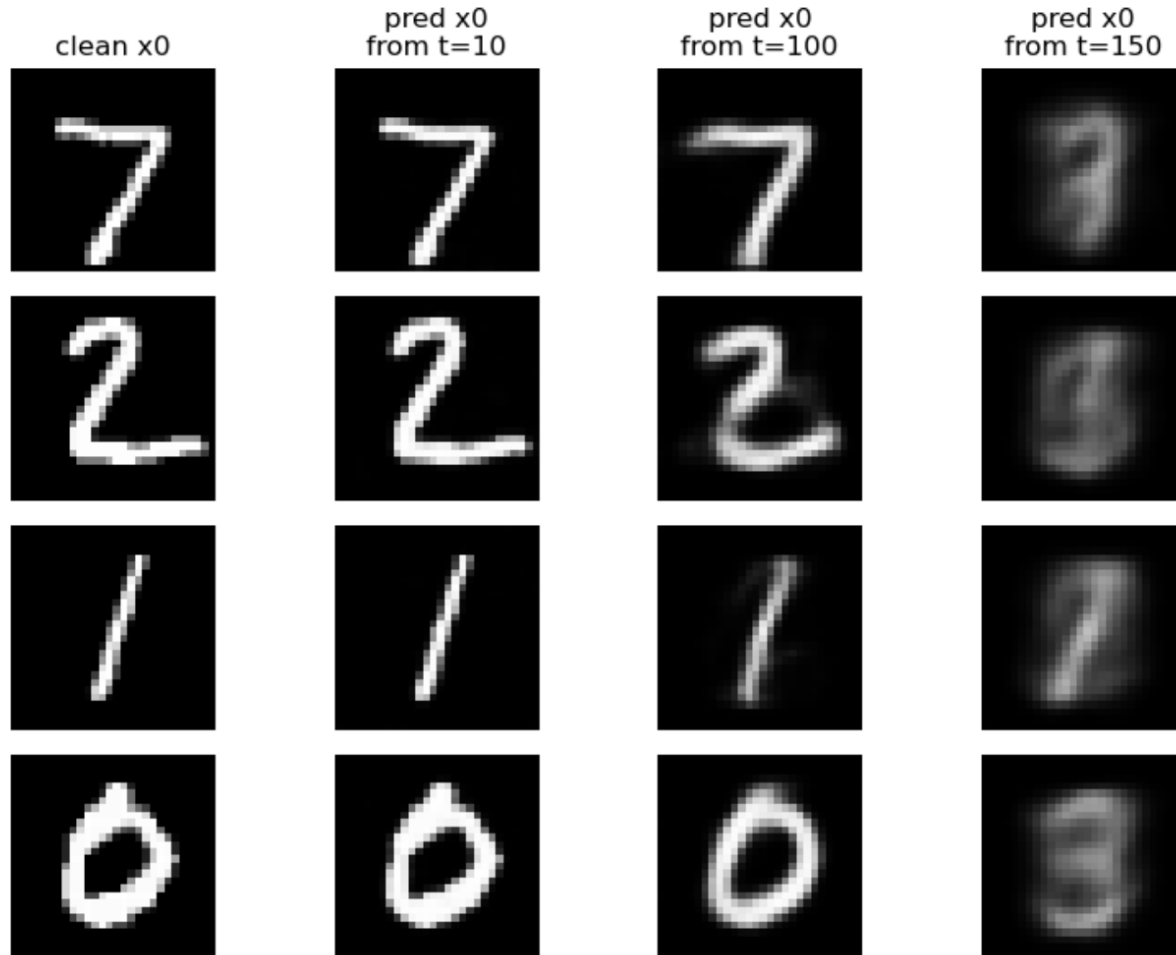


$$\sum_{t=1}^T \sum_{k=1}^K \|g_\theta(x_t, t) - x_0\|_2^2$$



Backward Process – $x_t \rightarrow x_0$

Direct clean-image prediction: useful, but harder as t grows





Backward Process – $x_t \rightarrow x_0$

$$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0.$$

$$g_\theta(x_t, t) \approx x_0,$$

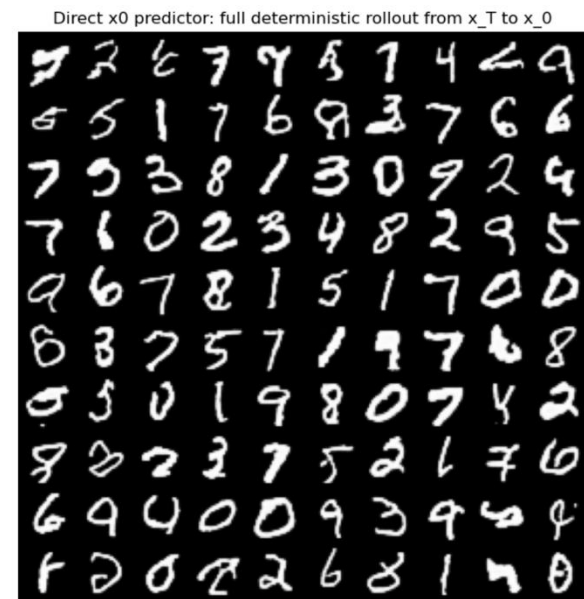
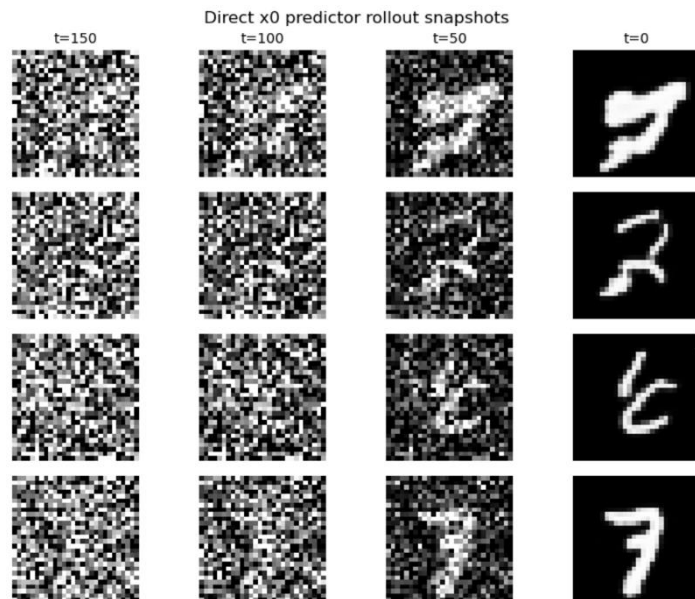
$$\hat{\epsilon}_t = \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}}.$$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

Predict x_0 = Predict noise!

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \hat{\epsilon}_t.$$





Backward Process – $x_t \rightarrow \sigma$

Given x_t , predict something that helps us move back toward data.

There are three possible targets:

$x_t \rightarrow x_{t-1}$	previous noisy state
$x_t \rightarrow x_0$	clean image
$x_t \rightarrow \epsilon$	noise that corrupted the image

Forward Process

cat $\longrightarrow$ c#a\$t%

Backward Process

cat $\longleftarrow$ c#a\$t%

What was the exact previous corrupted string?
Was it "c#a\$t%", "ca\$t%", "c#at%", ... ?

$$x_t \rightarrow x_{t-1}$$

What was the clean string?

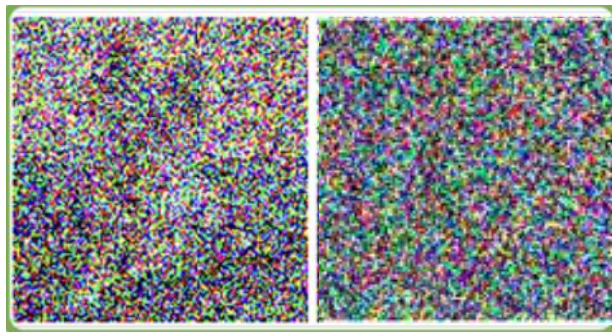
$$x_t \rightarrow x_0$$

Which symbols look like noise?
Answer: #, \$, %

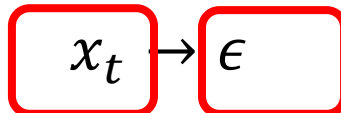
$$x_t \rightarrow \sigma$$



Backward Process – $x_t \rightarrow \sigma$



Current state



Noise

$$\{(x_t, \epsilon)\}_{t=1}^T$$

For each minibatch:

1. Sample clean images $x_0 \sim p_{\text{data}}(x)$.
2. Sample timesteps $t \sim \text{Uniform}\{1, \dots, T\}$.
3. Sample Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$.
4. Create the noisy image using the same closed form:

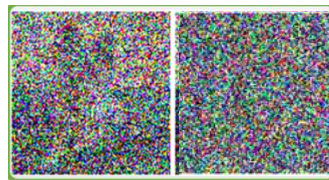
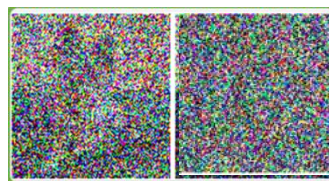
$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

5. Predict the noise inside the noisy image:

$$\hat{\epsilon} = \epsilon_{\theta}(x_t, t).$$

6. Train with noise-prediction loss:

$$\mathcal{L}_{\epsilon}(\theta) = \|\epsilon_{\theta}(x_t, t) - \epsilon\|^2.$$



...

